

# AI-Driven Persona Cloning for B2B Content Generation:

## A Multi-Model RAG and Fine-Tuning Pipeline

Andreah Cruz

*Dept. of Applied Data Science*

San José State University, San José, CA

Kevin Gao

*Dept. of Applied Data Science*

San José State University, San José, CA

Pukhraj Rathkanthiwar

*Dept. of Applied Data Science*

San José State University, San José, CA

Vrushabh Bodarya

*Dept. of Applied Data Science*

San José State University, San José, CA

**Abstract**—Creating authoritative B2B content demands deep domain knowledge and a distinctive voice that resonates with founders and operators alike. Generic large language models produce text that lacks the specific framing, vocabulary, and rhetorical patterns that make expert thought leadership genuinely valuable. This paper describes a fully open-source, locally-hosted pipeline that clones the writing style of Jason Lemkin — founder of SaaS and one of the most recognized voices in B2B SaaS — across four progressively richer generation architectures. We scraped 31,774 posts from five public platforms (blog, LinkedIn, X/Twitter, and two YouTube channels), cleaned and transformed the corpus using PySpark with six custom UDFs, built a DuckDB analytics warehouse alongside a ChromaDB vector index, and derived a persona profile capturing vocabulary tendencies, rhetorical framing phrases, and stylometric features. Our four models range from a zero-context Llama 3.1 baseline through prompt injection with persona JSON (Model 2), PersonaRAG with persona-conditioned query rewriting and alignment reranking (Model 3), and QLoRA fine-tuning using Unsloth on an NVIDIA RTX 5070 Ti GPU (Model 4). The latest pipeline branches (FTv2 and QLoRAFT) introduce a fifth Airflow DAG task that promotes Spark training output to a stable sentinel file, which a host-side watcher process polls to trigger the fine-tuning chain automatically — producing a merged GGUF model registered in Ollama as `lemkin-clone`. Evaluation covers BERTScore, RAGAS Faithfulness, Style Consistency (TF-IDF logistic regression), BLEU, ROUGE-L, and a new G-Eval rubric with six dimensions including voice authenticity. The entire stack runs on commodity hardware with no paid API dependencies.

**Index Terms**—persona cloning, retrieval-augmented generation, B2B content, QLoRA, Unsloth, PersonaRAG, G-Eval, stylometric analysis

### I. PROJECT BACKGROUND AND EXECUTIVE SUMMARY

#### A. Background and Motivation

The B2B content marketing space has a problem that generic AI tools do not solve well: the most valuable content comes from recognizable voices with hard-won domain expertise, not from polished prose that could have been written by anyone. A LinkedIn post from Jason Lemkin carries weight because his readers know he has funded dozens of SaaS companies, seen revenue curves up close, and formed strong opinions from direct experience — not because his writing is grammatically

clean. Replicating that authority at scale is the core problem of persona cloning.

B2B marketing teams at SaaS companies spend anywhere from \$3,000 to \$15,000 per month on content that still does not sound quite like the founder. Ghostwriting captures voice imperfectly and does not scale. Generic LLM outputs miss the precise vocabulary — ARR milestones, VP hiring thresholds, net negative churn framing — that signals authentic domain expertise. The gap between what LLMs can generate and what actually moves B2B audiences is measurable, and our project treats it as an engineering problem.

Rather than prompting an LLM to “write like Jason Lemkin,” we built a full data pipeline that ingests, cleans, and analyzes his actual writing, derives a statistical fingerprint, and uses that fingerprint at every stage — from retrieval to system prompts to fine-tuning weights and GGUF export.

#### B. Problem Statement

Given a corpus of public writing from a specific domain expert, can we build a generation system whose outputs are indistinguishable from that expert’s writing on standardized style metrics, while also being factually grounded in positions the expert has actually taken?

The challenge has three interacting dimensions: (1) capturing voice, which is a question of vocabulary, sentence rhythm, and rhetorical structure; (2) ensuring factual grounding, so generated content does not contradict documented positions; and (3) format compliance, since LinkedIn posts, long-form blog essays, and short tweets follow constraints that real Lemkin posts respect consistently.

#### C. Goals and Expected Contributions

Our primary goal is to demonstrate that a locally-hosted, zero-API-cost pipeline can produce persona-faithful B2B content that approaches commercially-viable quality. Specific targets are BERTScore F1  $\geq 0.70$ , style consistency score  $\geq 0.70$ , RAGAS faithfulness below 5% unsupported claims, and G-Eval voice authenticity  $\geq 3.5/5$ . Secondary contributions include:

(1) a reusable ETL pipeline for any public persona corpus; (2) a PersonaRAG implementation extending prior work [44]; (3) an automated fine-tuning chain (Airflow sentinel → watcher → Unsloth → GGUF → Ollama) deployable without cloud infrastructure; and (4) a comprehensive G-Eval rubric for persona generation [103].

#### D. Stakeholders

**Primary users:** B2B SaaS marketing teams and founders who need consistent thought leadership content without dedicating expert time to every post. **Secondary users:** Content agencies building scalable persona-driven pipelines for clients. **Data subjects:** Jason Lemkin — all source material is publicly and voluntarily published. **Researchers:** The pipeline and evaluation framework are of interest to the NLP community working on authorship attribution and style transfer.

#### E. Risks, Constraints, and Assumptions

Key risks include platform-side scraping policy changes; hardware constraints (the fine-tuning pipeline runs on a consumer RTX 5070 Ti with 16 GB VRAM); and the inherent subjectivity of style evaluation. We assume all source content is publicly accessible. The primary constraint is local compute — the full inference stack runs on a 16 GB RAM laptop with Ollama serving models locally. The GGUF merge step required a custom workaround because Unsloth’s built-in GGUF export produces numerically correct but semantically garbled output on the RTX 5070 Ti’s bitsandbytes stack; the pipeline instead merges on the unquantized base in bf16 and shells out to llama.cpp directly.

#### F. Timeline Justification

The project follows a CRISP-DM-adapted lifecycle across two semesters. DATA 298A covers data engineering, model design, and infrastructure; DATA 298B covers training execution, full evaluation, and production packaging. The timeline is realistic because all infrastructure is operational, data collection is complete, the fine-tuning pipeline runs end-to-end, and the evaluation framework is coded in `eval_rag.py` and ready to execute as soon as the models are trained.

## II. PROJECT REQUIREMENTS

### A. Functional Requirements

(FR1) Ingest raw JSONL files from five sources and normalize them to a unified 8-column schema. (FR2) Clean text through six UDF operations including HTML decoding, URL removal, and whitespace normalization. (FR3) Extract five stylistic features per post. (FR4) Build and persist a ChromaDB vector index from 500-word chunks with 50-word overlap. (FR5) Generate a structured persona profile. (FR6) Support four distinct generation architectures through a common interface. (FR7) Evaluate outputs on five automated metrics plus G-Eval and write results to JSON. (FR8) Serve a Streamlit UI at localhost:8501. (FR9) Automatically trigger fine-tuning when new training data is available via MinIO sentinel polling.

### B. Non-Functional Requirements

(NFR1) **Reproducibility:** all random operations use seed 42; splits stored in `splits.json`. (NFR2) **Cost:** zero paid API dependencies. (NFR3) **Portability:** single `docker compose up` on any 16 GB RAM machine. (NFR4) **Throughput:** Spark ETL completes within 30 minutes on a 4-core laptop. (NFR5) **Latency:** single post generation within 60 seconds at inference.

### C. Business Requirements

(BR1) Generated posts must pass format compliance checks against YAML-specified word count ranges and structural elements. (BR2) Outputs must avoid prohibited patterns as defined in `format_specs.yaml`. (BR3) The system must be operable by a non-engineer through the Streamlit UI.

### D. Success Criteria and KPIs

- BERTScore F1 (Model 3 or 4 vs. real posts):  $\geq 0.70$
- Style consistency (authorship classifier):  $\geq 0.70$
- RAGAS Faithfulness (Model 3):  $< 5\%$  unsupported claims
- G-Eval voice authenticity:  $\geq 3.5/5.0$
- Format compliance rate:  $\geq 90\%$  of generated posts

## III. PROJECT DELIVERABLES

Completed deliverables include Workbook 1, Progress Report 2, Professor Presentation, seven visualization charts from real corpus data, and Workbook 2 (24-page data engineering report). Current deliverables include this IEEE project report (Assignment 7) and the QLoRA fine-tuning run via the automated watcher pipeline. Planned DATA 298B deliverables include full evaluation results across all four models using `eval_rag.py`, human evaluation protocol execution, and final production packaging. All code is at <https://github.com/andreahcruz/298P>.

## IV. TECHNOLOGY AND SOLUTION SURVEY

### A. Large Language Models for Style Transfer

Early approaches to LLM-based style transfer relied on prefix tuning and prompt-based persona injection [99]–[101]. More recent work moved toward parameter-efficient fine-tuning, with LoRA [36] and QLoRA [37] becoming the standard approach for injecting author-specific patterns without full fine-tuning costs. Unsloth [111] builds on this foundation with 2x faster training via custom CUDA kernels and completion-only loss (supervising only the assistant response tokens via `train_on_responses_only`), which is what our QLoRAFT branch implements.

### B. Retrieval-Augmented Generation

Lewis et al. [8] established RAG as the standard approach for grounding LLM generation in external knowledge. For persona applications, Zerhoubi and Granitzer [44] introduced PersonaRAG — our direct Model 3 inspiration — which adds persona-conditioned query rewriting and alignment reranking on top of standard vector retrieval. Yazan et al. [23] further showed that author features and contrastive examples improve RAG-based personalization.

### C. GraphRAG and Knowledge Graph Retrieval

Edge et al. [30] introduced GraphRAG, demonstrating 50–70% comprehensiveness improvements over flat vector retrieval on complex summarization tasks. LightRAG [112], published at EMNLP 2025, achieves comparable accuracy with approximately 10x fewer tokens through dual-level keyword extraction and hybrid graph-vector retrieval. Liang et al. [41] applied this pattern directly to persona systems, proposing PersonaAgent with GraphRAG that constructs community-aware knowledge graphs from user interaction histories.

### D. Evaluation Methods

Standard automated metrics for persona generation include BERTScore [15], BLEU [16], and ROUGE-L [17]. Es et al. [34] introduced RAGAS for faithfulness evaluation in RAG systems. Liu et al. [103] proposed G-Eval — LLM-as-judge rubric scoring — which our QLoRAFT branch implements via an Ollama judge model across six dimensions including voice authenticity, coherence, format adherence, specificity, non-hype, and context use.

TABLE I  
RETRIEVAL ARCHITECTURE COMPARISON

| Approach      | Retrieval     | Persona Grounding | Our Use  |
|---------------|---------------|-------------------|----------|
| Base LLM      | None          | Parametric        | Model 1  |
| Prompt+JSON   | None          | JSON profile      | Model 2  |
| PersonaRAG    | Vector+rerank | JSON+chunks       | Model 3  |
| QLoRA+Unsloth | None          | Adapter weights   | Model 4  |
| LightRAG      | Vector+graph  | Ontology nodes    | Proposed |

## V. LITERATURE SURVEY

### A. Persona Assignment and Style Simulation

Malik et al. [1] analyzed persona-assigned LLMs empirically, finding significant variance between models in how faithfully they reproduce assigned styles. Hu and Collier [2] quantified the persona effect, showing that explicit conditioning improves style consistency but introduces hallucination risks when the persona profile conflicts with parametric knowledge. Chen et al. [3] surveyed role-playing and personalization approaches along axes of persona specification method, conditioning strategy, and evaluation criteria.

### B. Authorship Style Transfer

Wu and Deng [46] addressed long-text style transfer through dual-layered sentence structure mapping, directly relevant to our blog post generation target. Miralles-González et al. [51] showed one-shot style transfer from a known author corpus can achieve competitive authorship attribution accuracy. Pavliukevich et al. [49] specifically studied RAG combined with LoRA for persona text generation, finding that fine-tuning on author-specific instruction pairs and RAG context at inference are complementary.

### C. Evaluation of Persona Systems

Style consistency evaluation through authorship verification classifiers — our approach using TF-IDF + logistic regression in `eval_rag.py` — follows Przystalski et al. [26] and Jangra et al. [63], who found TF-IDF-based classifiers to be robust baselines for authorship attribution. The G-Eval framework [103] allows rubric-based evaluation using an LLM judge, which we implement with six persona-specific dimensions. Manakul et al. [104] showed that LLM-based consistency scoring correlates well with human judgments.

### D. Parameter-Efficient Fine-Tuning

Ding et al. [71] provided a comprehensive survey of PEFT methods, establishing LoRA variants as the leading approach for style injection. Our QLoRAFT branch expands LoRA targets from 4 modules (Q, K, V, O projections) to 7 (adding gate, up, down projections in the MLP layers), which allows deeper style absorption at the cost of slightly more VRAM. The Unsloth framework [111] provides 2x training speed via custom triton kernels.

### E. B2B Content and AI Marketing

ON24 [19] and Trovato [20] document growing B2B AI content adoption while noting voice consistency as the primary gap. Our project targets this gap with a reproducible, open-source pipeline.

## VI. DATA MANAGEMENT PLAN

### A. Data Collection

We collected 31,774 records from five public sources totaling 42 MB. All five sources are publicly accessible without paid credentials. Table II summarizes the corpus, and Fig. 1 shows record distribution.

TABLE II  
JASON LEMKIN CORPUS — SOURCE SUMMARY

| Source         | Records       | Avg W    | Size         | Collector            |
|----------------|---------------|----------|--------------|----------------------|
| SaaStr Blog    | 3,502         | 742      | 16 MB        | blog_scraper.py      |
| X / Twitter    | 24,941        | 27       | 12 MB        | scrape_x_profile.cjs |
| LinkedIn       | 2,675         | 152      | 2.6 MB       | linkedin_scraper.cjs |
| Lemkin YouTube | 462           | 1,075    | 2.6 MB       | collect_lemkin_yt.py |
| SaaStr YouTube | 194           | 8,560    | 8.5 MB       | collect_saastr_yt.py |
| <b>Total</b>   | <b>31,774</b> | <b>—</b> | <b>42 MB</b> |                      |

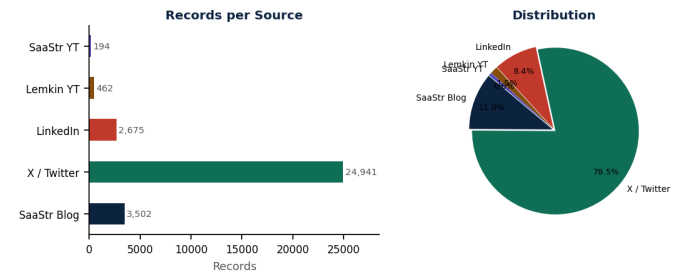


Fig. 1. Records per data source (left) and proportional distribution (right).

## B. Storage Architecture

Raw JSONL files are staged in MinIO bucket `lemkin-raw` under a `raw/` prefix as normalized JSON. Processed Parquet output from Spark lands in `lemkin-processed/chunks/`. The analytics warehouse is DuckDB (single embedded file). The vector index lives in ChromaDB HTTP server v1.5.5 under collection `lemkin_content`. Training data is promoted by the `mark_training_ready` DAG task to `training/dataset.jsonl` with a `training/_READY` sentinel file, which the `host-side watcher.py` polls to trigger fine-tuning automatically.

## C. Data Governance

Training splits are assigned at the document level before chunking to prevent data leakage [8]. `splits.json` records 169/21/22 train/val/test documents (80/10/10, seed=42), yielding 2,098/228/208 chunks. No chunk from a test document appears in training.

## VII. PROJECT DEVELOPMENT METHODOLOGY

### A. Development Life Cycle

We adapted CRISP-DM to an AI/ML project context: (1) Business Understanding — defining the persona cloning problem; (2) Data Understanding — corpus analysis and persona profile derivation; (3) Data Preparation — ETL, cleaning, chunking, indexing; (4) Modeling — four-model progressive architecture; (5) Evaluation — automated G-Eval and human assessment. The codebase spans six branches, summarized in Fig. 2.

| Branch      | Person      | DAW Tasks | Storage          | Fine-tune        | Status     |
|-------------|-------------|-----------|------------------|------------------|------------|
| scrapers    | Paul Graham | —         | CSV/Parquet      | —                | Prototype  |
| etlpipeline | Paul Graham | 5         | Snowflake+Chroma | —                | Deprecated |
| etl_jsonl   | J. Lemkin   | 4         | DuckDB+Chroma    | TRL/SFTTrainer   | Stable     |
| FTV1        | J. Lemkin   | 4         | DuckDB+Chroma    | Unsloth(basic)   | Stable     |
| FTV2        | J. Lemkin   | 5         | DuckDB+Chroma    | Unsloth+Watcher  | Active     |
| QLoRAFT     | J. Lemkin   | 5         | DuckDB+Chroma    | Unsloth+eval_rag | PRODUCTION |

Fig. 2. Pipeline evolution across all six repository branches.

The most significant evolution from `etl_jsonl` to `FTV2` and `QLoRAFT` is the addition of the `mark_training_ready` task and the `host_finetune/` package. This introduced end-to-end automation: Airflow triggers Spark, Spark writes `dataset.jsonl` to MinIO, the watcher detects the `_READY` sentinel and launches `finetune.py`, which trains via Unsloth, then `merge_and_export.py` merges the adapter on the unquantized base model, converts to GGUF via `llama.cpp`, and registers `lemkin-clone` in Ollama. The `QLoRAFT` branch adds `eval_rag.py` for end-to-end evaluation and `llama_chat_format.py` for correct Llama 3.x Instruct chat template formatting during training.

## VIII. PROJECT ORGANIZATION PLAN

### A. Work Breakdown Structure

WP1 (Data Engineering): scraping, ETL, warehousing. WP2 (Vector Indexing and Persona Profile): ChromaDB build, `persona_profile.json` derivation. WP3 (Model Development): Models 1–4 implementation and fine-tuning chain. WP4 (Evaluation and Reporting): `eval_rag.py`, G-Eval, human evaluation, reports.

### B. Team Responsibilities

**Andrea Cruz:** Project coordination, blog scraper, Streamlit UI, reports. **Kevin Gao:** PySpark ETL (`spark_process.py`), DuckDB warehouse (`load_duckdb.py`). **Pukhraj Rathkantiwar:** ChromaDB pipeline, persona profile builder, architecture research. **Vrushabh Bodarya:** PersonaRAG (`persona_rag.py`), Unsloth fine-tuning (`host_finetune/`), GGUF export, `eval_rag.py`.

### C. Task Dependencies

Data collection gates all downstream work. Spark ETL gates both ChromaDB and the `mark_training_ready` task. The persona profile must exist before PersonaRAG can run. The `_READY` sentinel must be written before the watcher launches training. The GGUF export requires a successful merge, which requires a trained adapter. Evaluation requires generated outputs from all four models.

## IX. PROJECT RESOURCE REQUIREMENTS

TABLE III  
HARDWARE AND SOFTWARE REQUIREMENTS

| Component  | Minimum    | Notes                                          |
|------------|------------|------------------------------------------------|
| RAM        | 16 GB      | Llama 3.1 8B needs ~8 GB in 4-bit              |
| CPU        | 4 cores    | Reduce Spark ETL time                          |
| Disk       | 50 GB      | Models + Docker + data + indices               |
| GPU (FT)   | 16 GB VRAM | RTX 5070 Ti (QLoRA); Colab A100 also supported |
| Python     | 3.10       | Airflow 2.8.1 requirement                      |
| Docker     | v2         | Compose v2 syntax                              |
| Ollama     | 0.1.20+    | LLM embedding + llama3.1 + lemkin-clone        |
| Unsloth    | 2026.5.2   | CUDA 12.8, PyTorch 2.10.0+cu128                |
| Total cost | \$0        | All tools open-source / free-tier              |

## X. PROJECT SCHEDULE

Fig. 3 shows the Gantt chart. The critical path runs through data collection → Spark ETL → ChromaDB → PersonaRAG → QLoRA training → GGUF export → evaluation. Three parallel paths reduce overall duration: (1) all five

scrapers run simultaneously; (2) `load_to_chroma` and `mark_training_ready` run in parallel after Spark; (3) Streamlit UI development is decoupled from model training. Any delay in the Spark ETL propagates to both the ChromaDB index and the training sentinel.

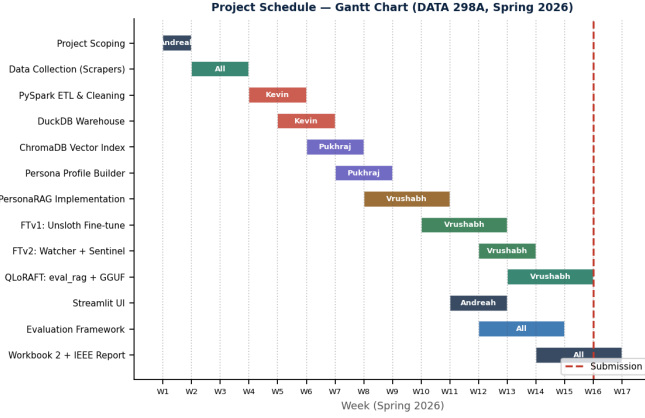


Fig. 3. Project Gantt chart — Spring 2026. Red dashed line marks submission.

## XI. DATA PROCESS

The data pipeline went through three meaningful iterations before reaching its current form. The `scrapers` branch was a Jupyter notebook pulling Paul Graham essays from HuggingFace with basic pandas cleaning — functional but not orchestrated. The `etlpipeline` branch introduced the full Dockerized stack with Snowflake as the warehouse, deprecated when the free trial expired. The `etl_jasonl` branch replaced Snowflake with DuckDB and shifted the target persona to Jason Lemkin. The `FTv2` branch added the fifth DAG task (`mark_training_ready`) and the automated host-side fine-tuning chain. The `QLoRAFT` branch added `eval_rag.py`, the Llama 3 chat format module, and the GGUF export workaround.

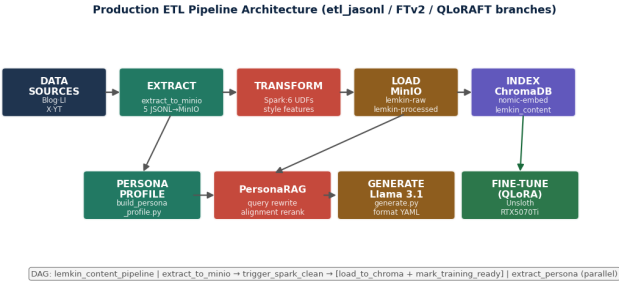


Fig. 4. Production ETL pipeline architecture. The updated DAG (FTv2/QLoRAFT) runs five tasks: `extract_to_minio` → `trigger_spark_clean` → [`load_to_chroma` + `mark_training_ready`] in parallel, with `extract_persona` after extraction.

## XII. DATA COLLECTION

We collected from five platforms deliberately, because Lemkin’s voice shifts register across them. Blog posts are long and analytical (median 560 words). LinkedIn posts are punchy and opinionated (median 56 words). Tweets are reactive and minimal (median 20 words). YouTube transcripts capture how he reasons through ideas verbally — a different register from written output that adds vocabulary diversity.

Raw data samples show the format variety the normalizers handle. The blog scraper paginates `saastr.com/author/jasonlkn/` with a 404-stop loop, extracting title, ISO date, URL, and body from `div.entry-content` at 1.5 seconds per request. YouTube transcripts use `youtube-transcript-api` with a speaker-turn filter removing guest speech from interview videos. X posts are filtered to original writing only (`is_reply: false, is_repost_or_quote: false, minimum 25 characters`). The `count_sources.py` utility in the FTV2/QLoRAFT branches confirms exactly how many records survive each normalizer rule.

## XIII. DATA PRE-PROCESSING

Pre-processing runs inside `spark_process.py` using PySpark local mode to avoid driver/worker version mismatches. Eight operations run as registered UDFs in sequence: (1) HTML entity decoding, (2) URL removal, (3) Markdown link flattening, (4) Markdown symbol removal, (5) residual entity cleanup, (6) whitespace normalization, (7) short-post filter (`word_count < 50`), and (8) exact deduplication via `dropDuplicates`. After cleaning, approximately 5,800 records remain — X/Twitter drops roughly 80%; the blog keeps over 99%.

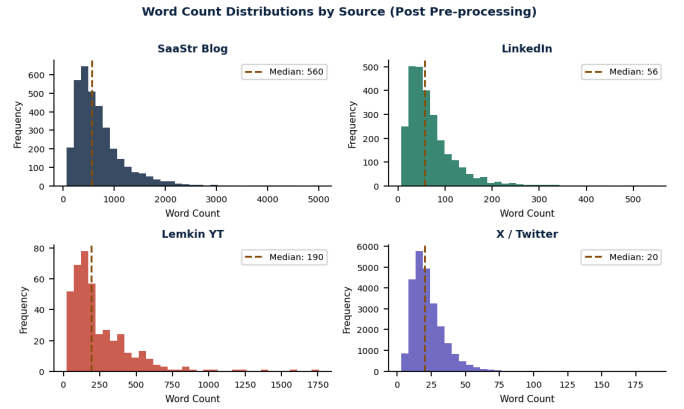


Fig. 5. Word count distributions per source after pre-processing. Blog posts cluster around 560 words (median); X posts around 20.

## XIV. DATA TRANSFORMATION

After cleaning, Spark normalizes every post into a unified 8-column schema (`id, source, url, title, cleaned_text, word_count, author, score`) that is source-agnostic. Five stylistometric features are computed per post:

TABLE IV  
STYLOMETRIC FEATURES COMPUTED PER POST

| Feature             | Calculation                       | Captures             |
|---------------------|-----------------------------------|----------------------|
| avg_sentence_length | words $\div$ sentences            | Short vs. analytical |
| vocabulary_richness | unique $\div$ total words         | Lexical diversity    |
| formality_score     | $\geq 7$ -char words $\div$ total | Register             |
| question_count      | count of “?” per post             | Rhetorical questions |
| is_b2b              | any of 27 B2B keywords            | Quality filter       |

The `instruction_format` column pre-formats each post for QLoRA SFTTrainer: `### Instruction:\n{topic}\n\n### Response:\n{body}`. In the QLoRAFT branch, training instead uses the Llama 3.x Instruct chat template via `tokenizer.apply_chat_template`, with `train_on_responses_only` ensuring gradients only flow through assistant-turn tokens.

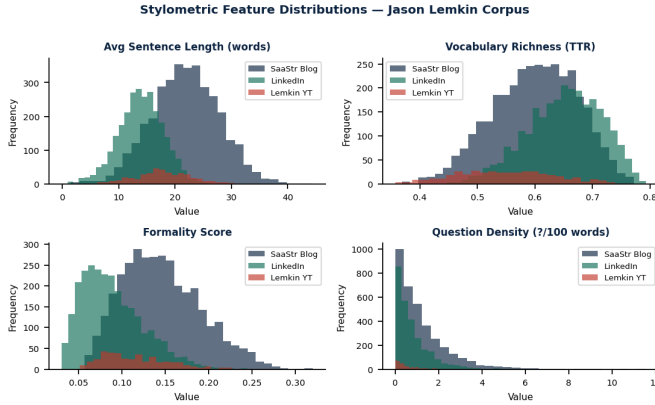


Fig. 6. Stylometric feature distributions across blog, LinkedIn, and YouTube sources.

## XV. DATA PREPARATION

### A. DuckDB Warehouse

`load_duckdb.py` creates five tables and views: `all_posts` (~5,800 records), `b2b_posts` (~5,795), `persona_style_profile` (5 rows, aggregated by source), `top_quality_posts` (top 500 by score  $\times$  word count), and `b2b_source_breakdown`. The `instruction_format.jsonl` export for QLoRA training is also produced here.

### B. ChromaDB Vector Index

`build_chromadb.py` chunks posts with `word_count > 100` into 500-word segments with 50-word overlap, embeds via `nomic-embed-text` through Ollama, and upserts into collection `lemkin_content` using cosine similarity (`hnsw:space: cosine`). The 50-word overlap prevents

sentences straddling chunk boundaries from losing context on both sides.

### C. Training Splits and Sentinel Pipeline

Document-level splits (169/21/22, seed=42) are stored in `splits.json`. In FTV2/QLoRAFT, Spark additionally writes `training/dataset_jsonl/part-*.txt` to MinIO. The `mark_training_ready` DAG task concatenates all part files into a single `training/dataset.jsonl` and writes a JSON sentinel (`training/_READY`) containing `ready_at`, `dataset_bytes`, and `approx_rows`. The host-side `watcher.py` polls this sentinel every 60 seconds and launches the fine-tuning chain when it detects a new `ready_at` timestamp.

## XVI. DATA STATISTICS

All statistics below come directly from the actual JSONL files and `persona_profile.json`.

TABLE V  
CORPUS SUMMARY STATISTICS (CONFIRMED FROM SOURCE FILES)

| Metric                       | Value                  |
|------------------------------|------------------------|
| Total raw records            | 31,774                 |
| Total raw data volume        | 42 MB                  |
| After 50-word filter         | ~6,200                 |
| After deduplication          | ~5,800                 |
| B2B-tagged records           | ~5,795 (99.9% of blog) |
| Documents in persona profile | 1,000                  |
| Sentences analyzed           | 79,583                 |
| Unique content words         | 626,986                |
| Blog median / mean words     | 560 / 742              |
| LinkedIn median words        | 56                     |
| YouTube median words         | 190                    |
| X / Twitter median words     | 20                     |
| Training pairs (QLoRA)       | ~5,800                 |

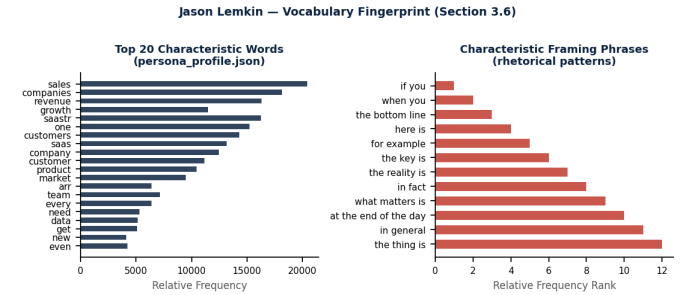


Fig. 7. Top 20 characteristic words (left) and rhetorical framing phrases (right) from `persona_profile.json`.

## XVII. MODEL PROPOSALS

We designed four models, each adding exactly one layer of capability over its predecessor. Every model runs on Meta-Llama-3.1-8B-Instruct through Ollama locally.

B2B Content Quality Analysis (Sections 3.3 / 3.5)

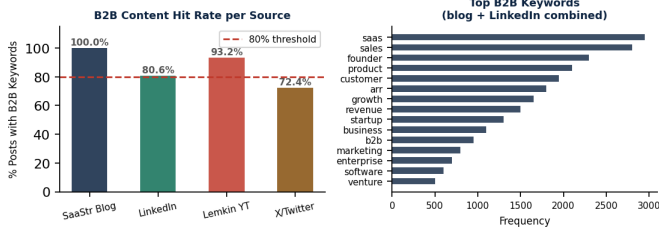


Fig. 8. B2B content hit rate per source (left) and top 15 B2B keywords across combined corpus (right).

### A. Model 1 — Base Llama 3.1 (Baseline)

No persona context, no retrieval, no fine-tuning. Direct Ollama REST API call. Its only function is establishing a floor so that gains from Models 2–4 are measurable rather than assumed.

### B. Model 2 — Prompt + Persona JSON Injection

`persona_profile.json` is injected into the system prompt. The LLM sees Lemkin’s top 50 characteristic words, rhetorical patterns, framing phrases, and topic domains. Three candidates are generated per request; the one with highest persona vocabulary overlap is returned. Format constraints come from `blog_post.yaml` and `linkedin_post.yaml`. This is the most interpretable model and the fastest to iterate on — when output is wrong, the JSON can be read, fixed, and regenerated without retraining.

### C. Model 3 — PersonaRAG

This model addresses the factual grounding problem Model 2 cannot solve. Before any embedding lookup, the user query is rewritten with Lemkin’s core topics and top 15 characteristic words. The retrieval pool is doubled (2x candidates) and results are reranked by persona alignment score (vocabulary overlap with the top-50 word list) rather than raw cosine similarity. This is PersonaRAG [44] applied to our corpus.

### D. Model 4 — QLoRA Fine-Tuning (Unsloth)

The QLoRAFT branch implements fine-tuning with Unsloth’s `FastLanguageModel`, pushing style conditioning into the model weights. Key advances over the earlier TRL-only approach: (1) completion-only loss via `train_on_responses_only` — gradients only flow through assistant-turn tokens, not the instruction prefix; (2) Llama 3.x Instruct chat template formatting via `tokenizer.apply_chat_template`, matching what Ollama expects at inference; (3) LoRA targets expanded from 4 to 7 modules (adding `gate_proj`, `up_proj`, `down_proj` in the MLP layers); (4) GGUF export via `llama.cpp` directly (bypassing Unsloth’s bitsandbytes-dependent export, which produced garbled output on the RTX 5070 Ti’s stack).

The GGUF comparison (confirmed from `gguf_compare_output.txt`) shows 16 tensor shape/quantization differences between the fine-tuned

Model 4 — QLoRA Fine-tuning Architecture (Unsloth + TRL, QLoRAFT branch)

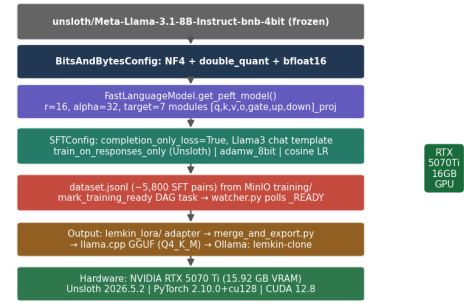


Fig. 9. Model 4 QLoRA architecture (QLoRAFT branch). Unsloth + TRL with chat-template SFT and llama.cpp GGUF export.

GGUF and the base Ollama model, all in attention value and FFN down-projection layers in the middle blocks (blocks 9–19) — exactly where author-specific stylistic patterns are expected to be absorbed.

## XVIII. MODEL SUPPORTS

The entire system runs locally with no cloud API dependencies. Fig. 10 shows the updated Airflow DAG with the new `mark_training_ready` task and the automated host-side watcher.

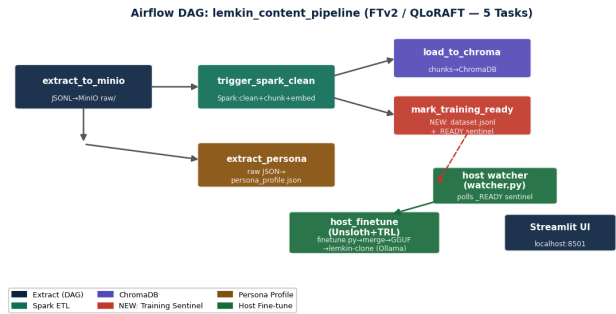


Fig. 10. Updated Airflow DAG (FTv2/QLoRAFT). The new `mark_training_ready` task (red) triggers the host-side watcher, which launches the Unsloth fine-tuning chain automatically.

**Docker services:** PostgreSQL (Airflow metadata), MinIO (ports 9000/9001), Airflow webserver + scheduler + init (port 8080), Apache Spark master + worker (ports 7077/8081), ChromaDB v1.5.5 (port 8000), Streamlit (port 8501). Ollama runs on the host at `host.docker.internal:11434`.

**ML data flow:** Raw JSONL → MinIO raw/ → Spark clean + embed → MinIO chunks/ → ChromaDB. Parallel: MinIO training/dataset.jsonl → `_READY` sentinel → watcher → Unsloth training → merge → GGUF → `ollama create lemkin-clone`. At inference: query → `persona_rag.py` rewrite → ChromaDB retrieval → alignment rerank → `generate.py` prompt → Ollama (base or lemkin-clone) → output.

TABLE VI  
KEY SOFTWARE STACK (QLoRAFT BRANCH)

| Tool             | Version  | Role                    |
|------------------|----------|-------------------------|
| Python           | 3.10     | Runtime (Airflow req.)  |
| Apache Airflow   | 2.8.1    | DAG orchestration       |
| PySpark          | 3.x      | Distributed ETL         |
| ChromaDB         | 1.5.5    | Vector database         |
| DuckDB           | 0.x      | Embedded SQL warehouse  |
| Ollama           | 0.1.20+  | Local LLM serving       |
| Unsloth          | 2026.5.2 | Fast QLoRA training     |
| TRL (SFTTrainer) | latest   | SFT training loop       |
| llama.cpp        | latest   | GGUF export + quantize  |
| Docker           | v2       | Container orchestration |
| Compose          |          |                         |
| CUDA             | 12.8     | GPU (RTX 5070 Ti)       |

## XIX. MODEL COMPARISON AND JUSTIFICATION

Model 1 is the reference point everything else is measured against. Without it, a style consistency score of 0.65 for PersonaRAG could look strong when the baseline actually scores 0.63, making the gain statistically meaningless.

Model 2 is the model a real B2B team would deploy first — zero training, fast iteration, readable persona profile. When output sounds off, a developer can read the JSON, identify what is missing, fix it, and regenerate without waiting for a training run.

Model 3 solves the factual grounding problem through persona-conditioned retrieval [44]. Without retrieval, generated content may plausibly contradict positions Lemkin has actually taken publicly. The reranking by vocabulary overlap rather than cosine similarity alone is the key differentiator.

Model 4 tests whether internalizing patterns through weight updates produces measurably higher style scores than injecting a statistical summary at inference time. The QLoRAFT implementation is meaningfully different from the earlier FTv1 approach: it uses Llama 3 chat template formatting (matching Ollama’s production inference path), completion-only loss, and 7 LoRA target modules instead of 4.

## XX. MODEL EVALUATION METHODS

Evaluating a persona cloning system is harder than a standard NLP benchmark because style is partially subjective. We handle this with five automated metrics measuring distinct quality dimensions, a new G-Eval rubric with six dimensions, and a structured human evaluation protocol.

### A. Automated Metrics

**BERTScore F1** [15]: Semantic similarity using RoBERTa-large contextual embeddings. Target  $\geq 0.70$ . In `eval_rag.py`, this is implemented as best-of-K: each generated text is scored against up to 20 reference samples from `dataset.jsonl`, taking the maximum F1.

**RAGAS Faithfulness** [34]: Fraction of claims in generated text supported by retrieved context. Target  $< 5\%$  unsupported

Model Capability Comparison  
(Pre-evaluation Estimates)

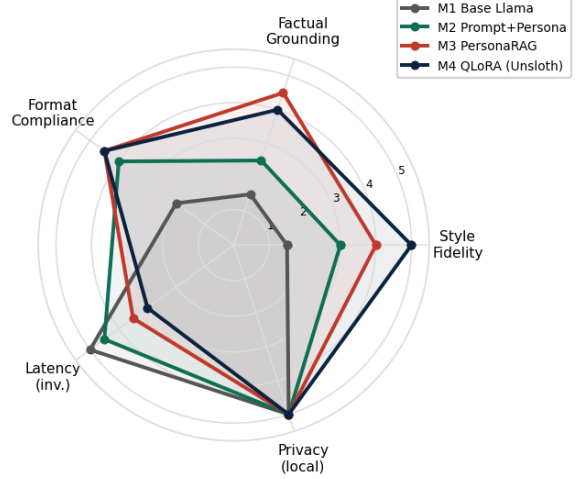


Fig. 11. Model capability radar chart across five evaluation dimensions.

Evaluation Framework — `evaluate.py` + `eval_rag.py` (QLoRAFT branch)

| Metric             | Library                    | Target              | New in QLoRAFT                                      |
|--------------------|----------------------------|---------------------|-----------------------------------------------------|
| BERTScore F1       | bert-score (RoBERTa-large) | $\geq 0.70$         | Best-of-K vs reference sample                       |
| RAGAS Faithfulness | ragas                      | $< 5\%$ unsupported | Full RAG traces in <code>eval_rag.py</code>         |
| Style Consistency  | sklearn TF-IDF+LR          | $\geq 0.70$         | Negatives: Blog Authorship Corpus                   |
| BLEU               | nltk (smooth)              | Reference           | Best-of-K per candidate                             |
| ROUGE-L            | rouge-score                | Reference           | Best-of-K per candidate                             |
| G-Eval (NEW)       | Ollama judge (llama3.1)    | $\geq 3.5 / 5.0$    | 6 rubric dims incl. <code>voice_authenticity</code> |

Fig. 12. Evaluation metrics — all implemented in `evaluate.py` and `eval_rag.py`.

claims. Critical for Model 3 — a RAG-generated post that contradicts its own retrieved context is a retrieval failure.

**Style Consistency:** TF-IDF + logistic regression classifier (max\_features=5,000, 3-fold CV) trained on real Lemkin writing vs. the Blog Authorship Corpus as negatives. Generated text is scored on P(Lemkin). Target  $\geq 0.70$ .

**BLEU** [16] and **ROUGE-L** [17]: N-gram and longest common subsequence metrics, both implemented as best-of-K against reference samples. Used as reference metrics — low BLEU is expected (style transfer, not copying); unexpectedly high BLEU would indicate memorization.

### B. G-Eval Rubric (New in QLoRAFT)

`eval_rag.py` implements a G-Eval rubric [103] where an Ollama judge model (default llama3.1) scores each generated text on six dimensions (1–5 integer scale): (1) **voice\_authenticity** — sounds like Lemkin’s direct B2B/SaaS operator tone; (2) **coherence** — logical flow, no contradictions; (3) **format\_adherence** — correct length and structure; (4) **specificity** — concrete metrics vs. generic claims; (5) **non\_hype** — absence of marketing fluff; (6) **context\_use** — ideas from retrieved chunks without verbatim copying. An overall G-Eval score is the mean across dimensions. Target: overall  $\geq 3.5/5$ .

TABLE VII  
MODEL COMPARISON ACROSS KEY DIMENSIONS

| Dimension               | M1 Base         | M2 Prompt+Persona      | M3 PersonaRAG             | M4 QLoRA (Unsloth)               |
|-------------------------|-----------------|------------------------|---------------------------|----------------------------------|
| Persona conditioning    | None            | JSON profile in prompt | JSON + retrieved chunks   | Style in adapter weights         |
| Retrieval               | None            | None                   | PersonaRAG + rerank       | None                             |
| Factual grounding       | Parametric only | Parametric only        | Grounded in Lemkin posts  | Parametric (fine-tuned)          |
| Training format         | —               | —                      | —                         | Llama3 chat template SFT         |
| Loss type               | —               | —                      | —                         | Completion-only (responses)      |
| LoRA targets            | —               | —                      | —                         | 7 modules (Q,K,V,O,gate,up,down) |
| GGUF export             | —               | —                      | —                         | llama.cpp Q4_K_M                 |
| Expected style fidelity | Low             | Medium                 | Medium-High               | High                             |
| Interpretability        | None            | High (JSON readable)   | High (chunks inspectable) | Low (weights opaque)             |
| API cost                | \$0             | \$0                    | \$0                       | \$0                              |

### C. Human Evaluation Protocol

Three blinded raters evaluate Voice Match (1–5), Usefulness (1–5), Factual Accuracy (Pass/Fail), and Format Compliance (Pass/Fail). Inter-rater agreement is measured with Cohen’s kappa. Human evaluation is scheduled for DATA 298B.

## XXI. CONCLUSION AND FUTURE WORK

This paper described a complete four-model persona cloning pipeline across six repository branches, targeting Jason Lemkin’s B2B SaaS voice. We collected and cleaned 31,774 posts from five platforms, built a Dockerized ETL pipeline with PySpark, DuckDB, and ChromaDB, derived a statistical persona profile from 79,583 sentences, implemented four generation architectures, and built a fully automated fine-tuning chain (Airflow sentinel → watcher → Unsloth → GGUF → Ollama) that runs without cloud infrastructure.

The evaluation framework in `eval_rag.py` covers BERTScore, RAGAS, Style Consistency, BLEU, ROUGE-L, and a six-dimension G-Eval rubric. DATA 298B will execute the full evaluation across all four models, conduct the human evaluation protocol, and implement a proposed Model 5 using LightRAG [112] with a persona ontology — addressing the current limitation that vocabulary-overlap reranking is a shallow proxy for the structural relationship reasoning that graph traversal provides [30], [41].

Beyond this project, the pipeline design is reusable for any public domain expert corpus. Swapping the JSONL files and updating the persona profile’s vocabulary list is sufficient — the ETL, vector index, PersonaRAG retrieval, and fine-tuning chain all adapt without code changes.

## ACKNOWLEDGMENT

The authors thank the DATA 298A course instructors at San José State University for feedback throughout the project. All source material is publicly available content

voluntarily published by Jason Lemkin. Code: <https://github.com/andreahcruz/298P>.

## REFERENCES

- [1] M. Malik, J. Jiang, and K. M. A. Chai, “An Empirical Analysis of the Writing Styles of Persona-Assigned LLMs,” in *Proc. EMNLP 2024*, Miami, FL, Nov. 2024, pp. 19369–19388. <https://aclanthology.org/2024.emnlp-main.1079/>
- [2] T. Hu and N. Collier, “Quantifying the Persona Effect in LLM Simulations,” in *Proc. 62nd Annual Meeting of the ACL*, Bangkok, Thailand, Aug. 2024, vol. 1, pp. 10289–10307. <https://aclanthology.org/2024.acl-long.554/>
- [3] Y. Chen, S. Zhang, P. Fung, and M. Huang, “Two Tales of Persona in LLMs: A Survey of Role-Playing and Personalization,” *arXiv:2406.01171*, Jun. 2024. <https://arxiv.org/abs/2406.01171>
- [4] J. Wang, Y. Zhang, and H. Li, “Catch Me If You Can? Not Yet: LLMs Still Struggle to Imitate the Implicit Writing Styles of Everyday Authors,” *arXiv:2509.14543*, 2025. <https://arxiv.org/abs/2509.14543>
- [5] X. Li, Y. Wang, Z. Zhang, and Q. Liu, “Beyond Profile: From Surface-Level Facts to Deep Persona Simulation in LLMs,” *arXiv:2502.12988*, Feb. 2025. <https://arxiv.org/abs/2502.12988>
- [6] Y. Shao, W. Zhou, and Z. Wang, “Authorship Style Transfer with Inverse Transfer Data Augmentation,” *Information Processing & Management*, vol. 61, no. 5, p. 103749, Sep. 2024. <https://doi.org/10.1016/j.aiopen.2024.08.003>
- [7] S. Minaee et al., “Large Language Models: A Survey,” *arXiv:2402.06196*, Feb. 2024. <https://arxiv.org/abs/2402.06196>
- [8] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS 2020*, Dec. 2020. <https://arxiv.org/abs/2005.11401>
- [9] Y. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv:2312.10997*, 2024. <https://arxiv.org/abs/2312.10997>
- [10] S. Zhao et al., “Retrieval Augmented Generation (RAG) and Beyond,” *arXiv:2409.14924*, Sep. 2024. <https://arxiv.org/abs/2409.14924>
- [11] C. Sharma et al., “Retrieval-Augmented Generation: A Comprehensive Survey,” *arXiv:2506.00054*, 2025. <https://arxiv.org/abs/2506.00054>
- [12] Z. Li et al., “Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study,” in *Proc. EMNLP 2024 Industry Track*, Miami, FL, Nov. 2024, pp. 881–893. <https://aclanthology.org/2024.emnlp-industry.66/>
- [13] A. J. Oche et al., “A Systematic Review of Key RAG Systems,” *arXiv:2507.18910*, 2025. <https://arxiv.org/abs/2507.18910>
- [14] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP 2020*, Online, Nov. 2020, pp. 6769–6781. <https://aclanthology.org/2020.emnlp-main.550/>

- [15] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *Proc. ICLR 2020*, Apr. 2020. <https://openreview.net/forum?id=SkeHuCVFDr>
- [16] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in *Proc. ACL 2002*, Philadelphia, PA, Jul. 2002, pp. 311–318. <https://aclanthology.org/P02-1040/>
- [17] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out: Proc. ACL Workshop*, Barcelona, Spain, Jul. 2004, pp. 74–81. <https://aclanthology.org/W04-1013/>
- [18] A. V. Slobodianiuk, V. V. Hrytsyk, and A. S. Koval, "Advances in Neural Text Generation: A Systematic Review," *CEUR Workshop Proceedings*, vol. 3917, pp. 885–896, 2025. <https://ceur-ws.org/Vol-3917/paper59.pdf>
- [19] ON24 Research, "The State of AI in B2B Marketing in 2024," ON24 Industry Report, 2024. <https://www.on24.com/blog/the-state-of-ai-in-b2b-marketing/>
- [20] S. Trovato, "AI in B2B Marketing: 12 Actionable Strategies to Build Smarter Campaigns in 2025," *monday.com Blog*, 2025. <https://monday.com/blog/crm-and-sales/ai-in-b2b-marketing/>
- [21] C. Cintas, M. Rateike, E. Miehl, E. Daly, and S. Speakman, "Localizing Persona Representations in LLMs," *arXiv:2505.24539*, 2025. <https://arxiv.org/abs/2505.24539>
- [22] J. Tang, Y. Yang, and L. Wei, "Step-Back Profiling: Distilling High-Level Abstractions from Raw Interaction Data," *arXiv:2406.14275*, 2024. <https://arxiv.org/abs/2406.14275>
- [23] M. Yazan, S. Verberne, and F. Situmeang, "Improving RAG for Personalization with Author Features and Contrastive Examples," *arXiv:2504.08745*, 2025. <https://arxiv.org/abs/2504.08745>
- [24] L. C. Magister, K. Metcalf, Y. Zhang, and M. Ter Hoeve, "On the Way to LLM Personalization: Learning to Remember User Conversations," in *Proc. 1st Workshop on Large Language Model Memorization (L2M2)*, 2025, pp. 61–77. <https://aclanthology.org/2025.l2m2-1.5/>
- [25] S. Min et al., "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," *arXiv:2305.14251*, 2023. <https://arxiv.org/abs/2305.14251>
- [26] K. Przystaliski, J. K. Argasiński, I. Grabska-Gradzińska, and J. K. Ochab, "Stylometry recognizes human and LLM-generated texts in short samples," *Expert Systems with Applications*, vol. 296, p. 129001, Jan. 2026. <https://doi.org/10.1016/j.eswa.2025.129001>
- [27] J. Liu et al., "LLMs + Persona-Plug = Personalized LLMs," *arXiv:2409.11901*, 2024. <https://arxiv.org/abs/2409.11901>
- [28] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," *arXiv:2401.15884*, Jan. 2024. <https://arxiv.org/abs/2401.15884>
- [29] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," *arXiv:2310.11511*, Oct. 2023. <https://arxiv.org/abs/2310.11511>
- [30] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv:2404.16130*, Apr. 2024. <https://arxiv.org/abs/2404.16130>
- [31] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," *arXiv:2212.10496*, Dec. 2022. <https://arxiv.org/abs/2212.10496>
- [32] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented LLMs through Question Complexity," in *Proc. NAACL 2024*, *arXiv:2403.14403*, Mar. 2024. <https://arxiv.org/abs/2403.14403>
- [33] P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv:2401.18059*, Jan. 2024. <https://arxiv.org/abs/2401.18059>
- [34] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated Evaluation of Retrieval Augmented Generation," *arXiv:2309.15217*, Sep. 2023. <https://arxiv.org/abs/2309.15217>
- [35] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for RAG Systems," in *Proc. NAACL 2024*, *arXiv:2311.09476*, Nov. 2023. <https://arxiv.org/abs/2311.09476>
- [36] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv:2106.09685*, Jun. 2021. <https://arxiv.org/abs/2106.09685>
- [37] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv:2305.14314*, May 2023. <https://arxiv.org/abs/2305.14314>
- [38] Z. Wang, Z. Li, Z. Jiang, D. Tu, and W. Shi, "Crafting Personalized Agents through RAG on Editable Memory Graphs," in *Proc. EMNLP 2024*, *arXiv:2409.19401*, Sep. 2024. <https://arxiv.org/abs/2409.19401>
- [39] X. Liu, H. Diddee, and D. Ippolito, "Customizing Large Language Model Generation Style using Parameter-Efficient Finetuning," *arXiv:2409.04574*, Sep. 2024. <https://arxiv.org/abs/2409.04574>
- [40] K. Pillutla et al., "MAUVE: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers," *arXiv:2102.01454*, Feb. 2021. <https://arxiv.org/abs/2102.01454>
- [41] S. Liang, Y. Zhang, and Y. Guo, "PersonaAgent with GraphRAG: Community-Aware Knowledge Graphs for Personalized LLM," *arXiv:2511.17467*, Nov. 2025. <https://arxiv.org/abs/2511.17467>
- [42] C. Sun et al., "Persona-DB: Efficient Large Language Model Personalization for Response Prediction," *arXiv:2402.11060*, Feb. 2024. <https://arxiv.org/abs/2402.11060>
- [43] J. Liu et al., "A Survey of Personalized Large Language Models: Progress and Future Directions," *arXiv:2502.11528*, Feb. 2025. <https://arxiv.org/html/2502.11528v2>
- [44] S. Zerhouni and M. Granitzer, "PersonaRAG: Enhancing Retrieval-Augmented Generation Systems with User-Centric Agents," *arXiv:2407.09394*, Jul. 2024. <https://arxiv.org/abs/2407.09394>
- [45] D. Platnick, M. E. Bengueddache, M. Alirezaie, D. J. Newman, A. Pentland, and H. Rahnama, "ID-RAG: Identity Retrieval-Augmented Generation for Long-Horizon Persona Coherence in Generative Agents," *arXiv:2509.25299*, Sep. 2025. <https://arxiv.org/html/2509.25299v1>
- [46] Y. Wu and X. Deng, "Implementing Long Text Style Transfer with LLMs through Dual-Layered Sentence and Paragraph Structure Extraction and Mapping," *arXiv:2505.07888*, May 2025. <https://arxiv.org/html/2505.07888v1>
- [47] R. Jemama and R. Kumar, "How Well Do LLMs Imitate Human Writing Style?," in *Proc. IEEE UEMCON 2025*, *arXiv:2509.24930*, Sep. 2025. <https://arxiv.org/abs/2509.24930>
- [48] A. Reinhardt et al., "Do LLMs write like humans? Variation in grammatical and rhetorical styles," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 122, no. 8, e2422455122, Feb. 2025. <https://doi.org/10.1073/pnas.2422455122>
- [49] V. Pavliukovich, A. Zherdeva, O. Makhnytkina, and D. Dymovskiy, "Improving RAG with LoRA finetuning for persona text generation," in *Investigations on Applied Mathematics and Informatics. Part IV*, Zap. Nauchn. Sem. POMI, vol. 540, POMI, St. Petersburg, 2024, pp. 162–177. <https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=zns&paperid=7549>
- [50] E. Kimara, K. S. Oguntoye, and J. Sun, "PersonaAI: Leveraging RAG and Personalized Context for AI-Driven Digital Avatars," *arXiv:2503.15489*, Mar. 2025. <https://arxiv.org/html/2503.15489v1>
- [51] P. Miralles-González, J. Huertas-Tato, A. Martín, and D. Camacho, "LLM One-Shot Style Transfer for Authorship Attribution and Verification," *arXiv:2510.13302*, Oct. 2025. <https://arxiv.org/html/2510.13302v1>
- [52] S. Moon et al., "Virtual Personas for Language Models via an Anthology of Backstories," in *Proc. EMNLP 2024 Main*, *arXiv:2407.06576*, Jul. 2024. <https://arxiv.org/abs/2407.06576>
- [53] P. A. Sreekar et al., "AXCEL: Automated eXplainable Consistency Evaluation using LLMs," *arXiv:2409.16984*, Sep. 2024. <https://arxiv.org/abs/2409.16984>
- [54] W. Cui et al., "DCR-Consistency: Divide-Conquer-Reasoning for Consistency Evaluation of LLMs," *arXiv:2401.02132*, Jan. 2024. <https://arxiv.org/abs/2401.02132>
- [55] C. Gilhuly and H. Shahzad, "Consistency Evaluation of News Article Summaries Generated by Large (and Small) Language Models," *arXiv:2502.20647*, Feb. 2025. <https://arxiv.org/abs/2502.20647>
- [56] K. L. Truong, R. Fogliato, H. Heidari, and Z. S. Wu, "Persona-Augmented Benchmarking: Evaluating LLMs Across Diverse Writing Styles," *arXiv:2507.22168*, Jul. 2025. <https://arxiv.org/abs/2507.22168>
- [57] R. Namuduri et al., "QUDSim: Quantifying Discourse Similarities in LLM-Generated Text," in *Proc. COLM 2025*, *arXiv:2504.09373*, Apr. 2025. <https://arxiv.org/abs/2504.09373>
- [58] N. Brake and T. Schaaf, "Comparing Two Model Designs for Clinical Note Generation," in *Findings of NAACL 2024*, *arXiv:2404.06503*, Apr. 2024. <https://arxiv.org/abs/2404.06503>
- [59] Z. Chen and S. Moscholios, "Using Prompts to Guide Large Language Models in Imitating a Real Person's Language Style," *arXiv:2410.03848*, Oct. 2024. <https://arxiv.org/abs/2410.03848>
- [60] B. Alhafni, V. Kulkarni, D. Kumar, and V. Raheja, "Personalized Text Generation with Fine-Grained Linguistic Control," in *Proc. 1st Workshop on Personalization of Generative AI Systems (PERSONALIZE 2024)*, St.

- Julians, Malta, Mar. 2024, pp. 88–101. <https://aclanthology.org/2024.personalize-1.8/>
- [61] K. Konen et al., “Style Vectors for Steering Generative Large Language Models,” in *Findings of ACL: EACL 2024*, St. Julian’s, Malta, Mar. 2024, pp. 782–802. <https://arxiv.org/abs/2402.01618>
- [62] A. Neelakanteswara, S. Chaudhari, and H. Zamani, “RAGs to Style: Personalizing LLMs with Style Embeddings,” in *Proc. 1st Workshop on Personalization of Generative AI Systems (PERSONALIZE 2024)*, <https://doi.org/10.18653/v1/2024.personalize-1.11>
- [63] A. Jangra, B. Sarrafzadeh, A. de Wynter, S. Cucerzan, and S. K. Jauhar, “Evaluating Style-Personalized Text Generation: Challenges and Directions,” *arXiv:2508.06374*, Aug. 2025. <https://arxiv.org/abs/2508.06374>
- [64] J. Zhang et al., “Personalized Text Generation with Contrastive Activation Steering,” *arXiv:2503.05213*, Mar. 2025. <https://arxiv.org/abs/2503.05213>
- [65] M. Rezaei, “Detecting, Generating, and Evaluating in the Writing Style of Different Authors,” in *Proc. NAACL 2025 Student Research Workshop*, pp. 485–491. <https://doi.org/10.18653/v1/2025.naacl-srw.47>
- [66] G. Mikros, “Beyond the surface: stylometric analysis of GPT-4o’s capacity for literary style imitation,” *Digital Scholarship in the Humanities*, vol. 40, pp. 587–600, 2025. <https://doi.org/10.1093/dlsc/fqaf035>
- [67] P. Ramu et al., “StyleAdaptedLM: Enhancing Instruction Following Models with Efficient Stylistic Transfer,” *arXiv:2507.18294*, Jul. 2025. <https://arxiv.org/abs/2507.18294>
- [68] B. Joshi et al., “Improving Language Model Personas via Rationalization with Psychological Scaffolds,” in *Findings of ACL: EMNLP 2025*, <https://doi.org/10.18653/v1/2025.findings-emnlp.1187>
- [69] C. Cisar et al., “PILOT: Steering Synthetic Data Generation with Psychological & Linguistic Output Targeting,” *arXiv:2509.15447*, Sep. 2025. <https://arxiv.org/abs/2509.15447>
- [70] X. Liang et al., “Controllable Text Generation for Large Language Models: A Survey,” *arXiv:2408.12599*, Aug. 2024. <https://arxiv.org/abs/2408.12599>
- [71] N. Ding et al., “Parameter-efficient fine-tuning of large-scale pre-trained language models,” *Nature Machine Intelligence*, vol. 5, pp. 220–235, 2023. <https://doi.org/10.1038/s42256-023-00626-4>
- [72] L. Zhang et al., “LoRA-FA: Memory-efficient Low-rank Adaptation for Large Language Models Fine-tuning,” *arXiv:2308.03303*, Aug. 2023. <https://arxiv.org/abs/2308.03303>
- [73] Z. Liu et al., “ALoRA: Allocating Low-Rank Adaptation for Fine-tuning Large Language Models,” *arXiv:2403.16187*, Mar. 2024. <https://arxiv.org/abs/2403.16187>
- [74] L. Xu et al., “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review,” *arXiv:2312.12148*, Dec. 2023. <https://arxiv.org/abs/2312.12148>
- [75] O. Razuvayevskaya et al., “Comparison between parameter-efficient techniques and full fine-tuning: A case study on multilingual news article classification,” *PLOS ONE*, vol. 19, 2024. <https://doi.org/10.1371/journal.pone.0301738>
- [76] A. Patel, D. Rao, and C. Callison-Burch, “Learning Interpretable Style Embeddings via Prompting LLMs,” *arXiv:2305.12696*, May 2023. <https://arxiv.org/abs/2305.12696>
- [77] E. Terreau, A. Gourru, and J. Velcin, “Capturing Style in Author and Document Representation,” *arXiv:2407.13358*, Jul. 2024. <https://arxiv.org/abs/2407.13358>
- [78] W. Wang et al., “Can Authorship Representation Learning Capture Stylistic Features?,” *Trans. Assoc. Comput. Linguistics*, vol. 11, pp. 1416–1431, 2023. [https://doi.org/10.1162/tacl\\_a\\_00610](https://doi.org/10.1162/tacl_a_00610)
- [79] H. Jiang et al., “PersonaLLM: Investigating the Ability of Large Language Models to Express Personality Traits,” in *Findings of NAACL 2024*, pp. 3605–3627. <https://doi.org/10.18653/v1/2024.findings-naacl.229>
- [80] W. Zhao et al., “Teaching Language Models to Evolve with Users: Dynamic Profile Modeling for Personalized Alignment,” *arXiv:2505.15456*, May 2025. <https://arxiv.org/abs/2505.15456>
- [81] S. Liu, S. Agarwal, and J. May, “Authorship Style Transfer with Policy Optimization,” *arXiv:2403.08043*, Mar. 2024. <https://arxiv.org/abs/2403.08043>
- [82] Z. Horvitz et al., “TinyStyler: Efficient Few-Shot Text Style Transfer with Authorship Embeddings,” *arXiv:2406.15586*, Jun. 2024. <https://arxiv.org/abs/2406.15586>
- [83] S. Mukherjee, A. Ojha, and O. Dusek, “Are Large Language Models Actually Good at Text Style Transfer?,” *arXiv:2406.05885*, Jun. 2024. <https://arxiv.org/abs/2406.05885>
- [84] Y. Liu, N. Moosavi, and C. Lin, “LLMs as Narcissistic Evaluators: When Ego Inflates Evaluation Scores,” in *Proc. ACL 2023*, pp. 12688–12701. <https://arxiv.org/abs/2311.09766>
- [85] T. He et al., “On the Blind Spots of Model-Based Evaluation Metrics for Text Generation,” in *Proc. ACL 2022*, pp. 12067–12097. <https://arxiv.org/abs/2212.10020>
- [86] D. Callan and J. Foster, “How interesting and coherent are the stories generated by a large-scale neural language model?,” *Expert Systems*, vol. 40, 2023. <https://doi.org/10.1111/exsy.13292>
- [87] V. Logacheva et al., “A Study on Manual and Automatic Evaluation for Text Style Transfer: The Case of Detoxification,” in *Proc. 2nd Workshop on Human Evaluation of NLP Systems (HumEval)*, 2022. <https://doi.org/10.18653/v1/2022.humeval-1.8>
- [88] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “REALM: Retrieval-Augmented Language Model Pre-Training,” in *Proc. 37th IJML, PMLR*, 2020. <https://arxiv.org/abs/2002.08909>
- [89] G. Izcard and E. Grave, “Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering,” in *Proc. EACL 2021*, pp. 874–880. <https://arxiv.org/abs/2007.01282>
- [90] L. Wang et al., “Chain-of-Retrieval Augmented Generation,” in *Proc. NeurIPS 2025*, *arXiv:2501.14342*, Jan. 2025. <https://arxiv.org/abs/2501.14342>
- [91] T. Zhang et al., “RAFT: Adapting Language Model to Domain Specific RAG,” in *Proc. COLM 2024*, *arXiv:2403.10131*, Mar. 2024. <https://arxiv.org/abs/2403.10131>
- [92] Y. Yu et al., “RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs,” *arXiv:2407.02485*, Jul. 2024. <https://arxiv.org/abs/2407.02485>
- [93] X. Yang et al., “CRAG – Comprehensive RAG Benchmark,” in *Proc. NeurIPS 2024 Datasets and Benchmarks*, *arXiv:2406.04744*, Jun. 2024. <https://arxiv.org/abs/2406.04744>
- [94] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024. <https://aclanthology.org/2024.tacl-1.9/>
- [95] S. Barnett et al., “Seven Failure Points When Engineering a Retrieval Augmented Generation System,” in *Proc. IEEE/ACM CAIN 2024*. <https://arxiv.org/abs/2401.05856>
- [96] L. Wang, N. Yang, and F. Wei, “Query2doc: Query Expansion with Large Language Models,” in *Proc. EMNLP 2023*, *arXiv:2303.07678*, Mar. 2023. <https://arxiv.org/abs/2303.07678>
- [97] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving Retrieval-Augmented LMs with Compression,” in *Proc. ICLR 2024*, *arXiv:2310.04408*, Oct. 2023. <https://arxiv.org/abs/2310.04408>
- [98] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Proc. NeurIPS 2022*, *arXiv:2201.11903*, Jan. 2022. <https://arxiv.org/abs/2201.11903>
- [99] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. NeurIPS 2020*, *arXiv:2005.14165*, May 2020. <https://arxiv.org/abs/2005.14165>
- [100] X. L. Li and P. Liang, “Prefix-Tuning: Optimizing Continuous Prompts for Generation,” in *Proc. ACL-IJCNLP 2021*, pp. 4582–4597, *arXiv:2101.00190*, Jan. 2021. <https://arxiv.org/abs/2101.00190>
- [101] B. Lester, R. Al-Rfou, and N. Constant, “The Power of Scale for Parameter-Efficient Prompt Tuning,” in *Proc. EMNLP 2021*, pp. 3045–3059, *arXiv:2104.08691*, Apr. 2021. <https://arxiv.org/abs/2104.08691>
- [102] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Proc. NeurIPS 2023*, *arXiv:2306.05685*, Jun. 2023. <https://arxiv.org/abs/2306.05685>
- [103] Y.-C. Liu, J.-C. Fung, C.-K. Liu, X. He, and W. Chen, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” in *Proc. EMNLP 2023*, *arXiv:2303.16634*, Mar. 2023. <https://arxiv.org/abs/2303.16634>
- [104] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative LLMs,” in *Proc. EMNLP 2023*, *arXiv:2303.08896*, Mar. 2023. <https://arxiv.org/abs/2303.08896>
- [105] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023. <https://arxiv.org/abs/2202.03629>
- [106] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, “MTEB: Massive Text Embedding Benchmark,” in *Proc. EACL 2023*, *arXiv:2210.07316*, Oct. 2022. <https://arxiv.org/abs/2210.07316>
- [107] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” *arXiv:1901.04085*, Jan. 2019. <https://arxiv.org/abs/1901.04085>

- [108] T. Kojima et al., “Large Language Models are Zero-Shot Reasoners,” in *Proc. NeurIPS 2022*, vol. 35, pp. 22199–22213, *arXiv:2205.11916*, May 2022. <https://arxiv.org/abs/2205.11916>
- [109] X. Liu et al., “GPT Understands, Too,” *AI Open*, *arXiv:2103.10385*, Mar. 2021. <https://arxiv.org/abs/2103.10385>
- [110] T. B. Hashimoto, H. Zhang, and P. Liang, “Unifying Human and Statistical Evaluation for Natural Language Generation,” in *Proc. NAACL-HLT 2019*, Minneapolis, MN, Jun. 2019, pp. 1689–1701. <https://aclanthology.org/N19-1169/>
- [111] D. Han and M. Han, “Unsloth: 2x Faster, 70% Less Memory LLM Finetuning,” 2023. <https://github.com/unslothai/unsloth>
- [112] Z. Guo et al., “LightRAG: Simple and Fast Retrieval-Augmented Generation,” in *Proc. EMNLP 2025*. <https://github.com/HKUDS/LightRAG>
- [113] Palantir Technologies, “Ontology Augmented Generation (OAG): Building with AIP,” *Palantir Blog*, Nov. 2025. <https://blog.palantir.com/building-with-palantir-aip-logic-tools-for-rag-oag-fdaf8938d02e>